Bases de Datos I

Torneo Interfacultades de Fútbol 5

Informe del proyecto



Autores: Trinidad Bunge, Maximiliano López y Paulina Silveira

Universidad Católica del Uruguay

Docentes: Gabriel Felipe, Marcelo Arrarte

Segundo semestre de 2026

Índice

Arquitectura general **3**
Modelo de datos **4**
Cambios respecto a la estructura de la letra 5
Decisiones de tipos 6
Normalización 6
El resultado no se guarda 7
Reglas de negocio: dónde se valida cada una **7**
Planteles 8
Partidos 8
Planillas, goles y tarjetas 8
Sanciones 9
Validación en las tres capas 9
Procedimientos, vistas y reportes **10**
Procedimientos almacenados 10
Vistas 10
Reportes 10
Seguridad **12**
Usuarios por rol 12
Login y tokens 13
Auditoría 13
Otras medidas 13
Backend, frontend y pruebas **14**
Backend 14
Frontend 14
Pruebas automatizadas 16
Mejoras implementadas y consideradas **16**
Implementadas 16
Consideradas y no implementadas 17
Limitaciones y vulnerabilidades conocidas **17**
Conclusiones **19**
Bitácora **19**
Reparto del trabajo 21
Reflexiones finales 21
Referencias **22**
Declaración del uso de inteligencia artificial **22**

Torneo Interfacultades de Fútbol 5: informe del proyecto

Para este obligatorio teníamos que construir un sistema que gestionara el Torneo Interfacultades de Fútbol 5: las ediciones, los equipos, el fixture, las planillas de cada partido y las sanciones. La decisión que más marcó el trabajo fue que las reglas del torneo tenían que cumplirse en la propia base de datos, sin importar si alguien entra desde la aplicación, desde DataGrip o desde otra herramienta.

En este informe explicamos cómo armamos el modelo de datos, dónde validamos cada regla y cómo cuidamos la integridad y la seguridad de la información. También contamos qué mejoras hicimos respecto a la estructura de la letra, qué limitaciones conocemos y cómo nos organizamos como equipo. El código, el diagrama y las instrucciones para ejecutar el sistema están en el repositorio del proyecto: https://github.com/maaxilopp/Obligatorio-BD1.

Arquitectura general

El sistema tiene tres partes. La primera es la base de datos en MySQL, que guarda la información del torneo y asegura su integridad con restricciones, triggers y procedimientos almacenados.

La segunda es el backend, hecho en Python con FastAPI. Comunica la base con la aplicación: recibe las solicitudes, valida los datos con Pydantic, una librería de Python que valida los datos que llegan a la API antes de que el backend los use, autentica a los usuarios y controla los permisos según su rol. Como pide la letra, las consultas a MySQL se escriben directamente en SQL, sin usar un ORM. Para conectarnos usamos MySQL Connector/Python.

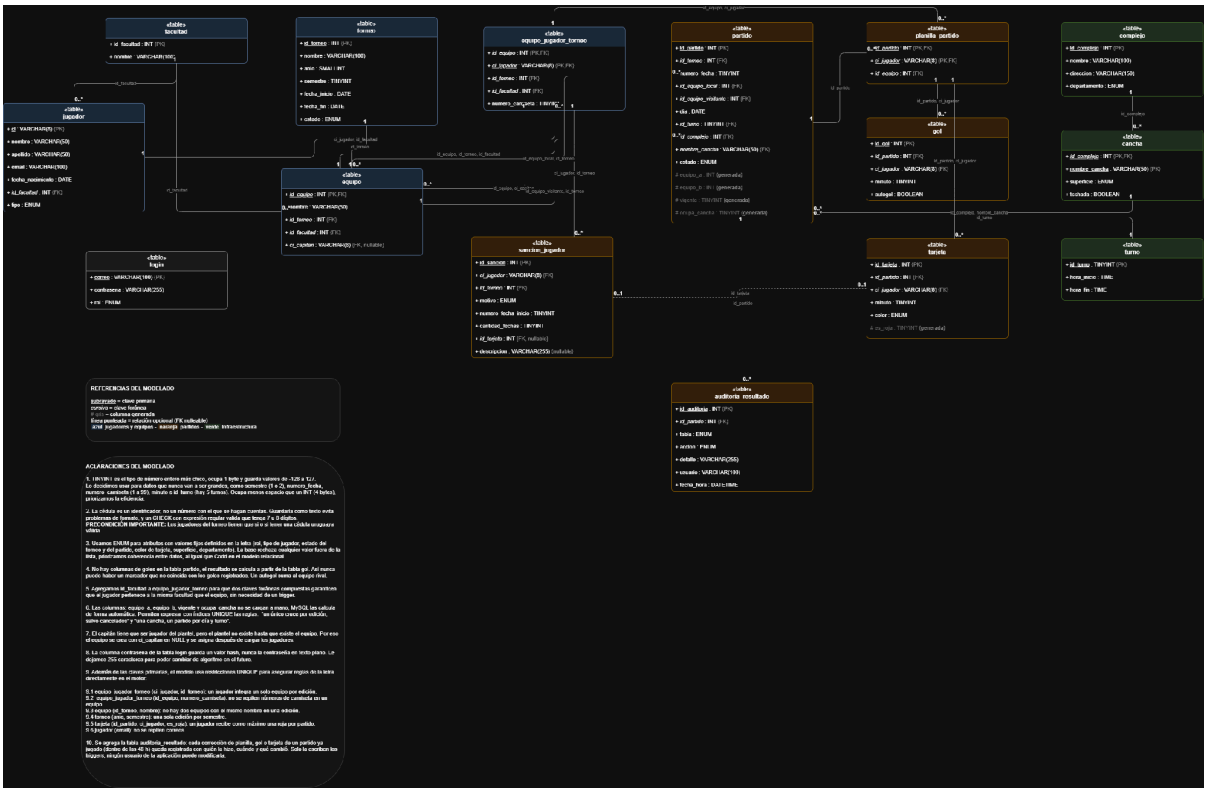
La tercera es el frontend, hecho con React y Vite. Es la interfaz que usan las personas: incluye el inicio de sesión, la navegación entre secciones y las pantallas de jugadores, equipos, partidos, planillas, sanciones y reportes.

Además, como se recomendaba en la consigna, usamos Docker Compose para levantar MySQL y el backend con un solo comando, y pytest para automatizar las pruebas.

Modelo de datos

El modelo tiene las 14 tablas que pide la letra y una tabla más, auditoria_resultado, para registrar las correcciones de resultados. Las tablas organizan la información de las distintas ediciones del torneo: facultades, jugadores, equipos, partidos, canchas, goles, tarjetas y sanciones. Las relaciones se definen con claves primarias y foráneas, y las restricciones del esquema controlan que los datos sean válidos. El esquema está en el repositorio en la siguiente ubicación: db/01_schema.sql y la Figura 1 muestra el diagrama completo.

Figura 1 - Diagrama del modelo de datos



El archivo en tamaño completo está en el repositorio en la siguiente ubicación: docs/diagrama_bd1.drawio.png

Cambios respecto a la estructura de la letra

Durante el desarrollo le hicimos algunos cambios a la estructura propuesta, para mejorar la integridad de los datos y hacer más fácil cumplir las reglas del torneo.

El primero fue agregar id_facultad a la tabla equipo_jugador_torneo. Con ese atributo pudimos definir claves foráneas compuestas que conectan al equipo y al jugador con su facultad, y así la base garantiza, sin triggers, que todos los integrantes de un equipo sean de la facultad que representa.

En la tabla partido agregamos cuatro columnas generadas: equipo_a, equipo_b, vigente y ocupa_cancha. MySQL las calcula solo y nos permiten usar restricciones UNIQUE para dos reglas, que dos equipos no se enfrenten más de una vez en la misma edición, salvo que el partido se haya cancelado, y que una cancha no tenga dos partidos en el mismo día y turno. Lo que hacen vigente y ocupa_cancha es quedar nulos cuando el partido no debe contar, porque una restricción UNIQUE ignora esos valores.

Con la misma idea agregamos la columna generada es_roja en la tabla tarjeta, que impide que un jugador reciba dos rojas en un partido.

En sancion_jugador sumamos id_tarjeta y descripcion. El primero conecta una sanción automática con la tarjeta que la generó, así que si se borra esa tarjeta por una corrección, la base borra la sanción en cascada. El segundo guarda información adicional de la sanción.

También permitimos que ci_capitan quede temporalmente en NULL en la tabla equipo, porque primero se crea el equipo y después se cargan sus jugadores. Antes de pasar el torneo a “en curso” se verifica que cada equipo tenga un capitán de su plantel, y una vez iniciado ningún equipo puede quedar sin capitán.

Por último, la tabla auditoria_resultado registra las correcciones de resultados, quién hizo el cambio, cuándo y qué se modificó.

Decisiones de tipos

Para elegir los tipos de datos miramos qué representa cada atributo y qué restricciones pide la letra. La cédula se guarda como texto porque es un identificador y no un número con el que se hagan cuentas, y una restricción de chequeo con una expresión regular verifica que tenga 7 o 8 dígitos.

Cuando un atributo solo puede tomar algunos valores, como el rol del usuario, el tipo de jugador, el estado del torneo, el color de la tarjeta, la superficie de la cancha o el departamento, usamos enums con las opciones correspondientes. Así la base rechaza cualquier opción que no corresponda.

Para los números chicos, como el semestre, el número de fecha, el número de camiseta, el minuto y el turno, usamos tinyint, porque son valores chicos y enteros. Cuando hace falta, una restricción de chequeo limita el rango, por ejemplo, el minuto de un gol o de una tarjeta tiene que estar entre 0 y 60, que es la duración de un partido.

Todas las claves foráneas están definidas de forma explícita. Cuando dos datos tienen que ser de la misma edición del torneo, usamos claves compuestas para que no se puedan mezclar ediciones.

Normalización

Al diseñar el modelo buscamos que cada dato se guarde en un solo lugar, para que no pueda quedar distinto en dos tablas. Por ejemplo, los datos de un jugador están únicamente en la tabla jugador, y el plantel solo relaciona al jugador con su equipo.

Hay dos excepciones que hicimos a propósito. En equipo_jugador_torneo, id_torneo e id_facultad se podrían deducir de id_equipo. En partido pasa algo parecido, porque id_torneo se puede deducir de los equipos que juegan. Decidimos mantener esas columnas porque son las que permiten armar las claves foráneas compuestas. Gracias a ellas, la base garantiza que un jugador sea de la facultad de su equipo y que los dos equipos de un partido sean de la misma edición. Preferimos esa pequeña repetición antes que resolver esas reglas con más triggers, y no hay riesgo de que los datos se contradigan, porque las propias claves foráneas lo impiden.

Las columnas generadas tampoco son datos repetidos en este sentido, ya que nadie las carga a mano, MySQL las calcula a partir de los demás datos de la fila.

El resultado no se guarda

Decidimos no guardar el resultado de los partidos en la tabla partido. El marcador se calcula a partir de los goles registrados en la tabla gol, sumando los autogoles al equipo contrario. Así evitamos tener el mismo dato dos veces y el resultado siempre coincide con los goles cargados. Si hay que corregir un resultado, alcanza con corregir los goles. Este cálculo lo hace la vista v_resultado_partido, que también usan los reportes.

Reglas de negocio: dónde se valida cada una

Las reglas que se pueden expresar directamente en el esquema se controlan con claves foráneas y restricciones de chequeo o UNIQUE. Para las más complejas, que dependen de otros registros o del momento en que se hace la operación, usamos triggers y procedimientos almacenados. Cuando una regla no se cumple, el trigger rechaza la operación con un mensaje y el backend se lo muestra al usuario.

Planteles

Que todos los jugadores sean de la facultad del equipo y que un jugador no integre dos equipos en la misma edición se garantiza con claves foráneas compuestas y restricciones UNIQUE. El máximo de 10 jugadores, el máximo de 2 docentes o funcionarios y que los planteles solo se modifiquen durante la inscripción se controlan con triggers. El mínimo de 5 jugadores se verifica al programar un partido y al iniciar el torneo, porque mientras se arma el plantel es normal tener menos. El capitán se verifica al iniciar el torneo.

Partidos

Las claves foráneas compuestas aseguran que los dos equipos sean de la misma edición, y una restricción de chequeo impide que un equipo juegue contra sí mismo. Las restricciones UNIQUE sobre columnas generadas evitan que dos equipos se enfrenten dos veces, salvo que el partido se haya cancelado, y que una cancha tenga dos partidos en el mismo día y turno. Que un equipo no juegue dos partidos en la misma fecha y que el partido esté dentro de las fechas del torneo se valida con triggers. Un partido jugado no se puede eliminar ni modificar.

Planillas, goles y tarjetas

Las claves foráneas aseguran que en la planilla solo figuren jugadores del plantel y que solo hagan goles o reciban tarjetas quienes están en ella. Los triggers impiden cargar a un jugador suspendido, marcar como jugado un partido que todavía no se disputó y corregir un resultado pasadas las 48 horas.

También controlamos las expulsiones. Un jugador expulsado, ya sea por roja directa o por doble amarilla, no puede tener goles ni tarjetas después del minuto de su expulsión, y nadie puede recibir tres amarillas en un partido.

Esto se cumple sin importar en qué orden se carguen los datos: si se carga primero la amarilla del minuto 20 y después la del minuto 10, la segunda amarilla es la del 20, y la base rechaza la carga si el jugador ya tiene un gol en un minuto posterior.

Sanciones

Las tarjetas generan sanciones de forma automática. Una tarjeta roja directa implica una fecha de suspensión. También decidimos aplicar esta sanción cuando un jugador recibe dos tarjetas amarillas en el mismo partido, siguiendo las reglas habituales del fútbol. Lo mismo ocurre cuando un jugador acumula tres amarillas durante una edición del torneo.

Si en un mismo partido coinciden una doble amarilla y la acumulación de tres amarillas, decidimos sumar las sanciones. En ese caso, el jugador cumple dos fechas consecutivas de suspensión: primero la correspondiente a la doble amarilla y después la de acumulación.

Si se corrige una tarjeta, las sanciones se recalculan. Por ejemplo, si se borra una de las dos amarillas de una doble amarilla, esa sanción desaparece. El administrador también puede cargar sanciones disciplinarias, como por ejemplo la mordida de Luis Suárez en el Mundial de 2014 en el partido contra Italia, que no dependen de una tarjeta y no pueden abarcar fechas en las que el jugador ya figura en una planilla.

Validación en las tres capas

Las validaciones están en las tres capas del sistema. El frontend comprueba los datos de los formularios antes de enviarlos, el backend los valida con Pydantic y la base verifica las reglas antes de guardar los cambios. De esta manera el usuario recibe un aviso rápido en la pantalla, pero la garantía final está en la base.

Procedimientos, vistas y reportes

Procedimientos almacenados

El procedimiento marcar_jugado(id_partido) cambia el estado del partido a jugado. Se ejecuta con los permisos de quien lo creó, y por eso el planillero puede cerrar un partido sin tener permiso para modificar la tabla partido.

El procedimiento generar_fixture(id_torneo, primer_dia) arma un fixture de todos contra todos con el método circular: un equipo queda fijo y los demás rotan una posición en cada fecha, de modo que todos se enfrentan una sola vez. Programa una fecha por semana y le asigna a cada partido la primera combinación libre de cancha y turno.

De forma complementaria, hay procedimientos auxiliares, como validar_partido, validar_cupos_plantel, validar_expulsion, recalcular_sanciones_amarillas y registrar_auditoria. Concentran validaciones que usan varios triggers, para no repetir código.

Vistas

Las vistas guardan los cálculos que se repiten, v_gol_detalle, v_resultado_partido, v_partido_equipo, v_tabla_posiciones, v_goleadores, v_fair_play y v_sanciones. Los reportes y el backend se apoyan en ellas, lo que evita repetir lógica y hace que la aplicación y DataGrip muestren lo mismo.

Reportes

Los nueve reportes de la letra están en db/07_reportes.sql. Casi todos se pueden consultar desde la API en rutas del tipo /torneos/{id_torneo}/posiciones, y el de jugadores por facultad está en /reportes/jugadores-facultad. Los reportes son los siguientes:

- **Tabla de posiciones:** usa RANK() ordenando por puntos, diferencia de goles y goles a favor.

- **Goleadores:** muestra el top 10 sin contar autogoles. Usa DENSE_RANK() para la posición y RANK() para el corte, así que si hay un empate en el puesto 10 entran todos los empatados, nuevamente una decisión de diseño por nuestra parte.
- **Fair play por equipo:** cada amarilla vale 1 punto y cada roja 3, y gana el equipo con menor puntaje.
- **Promedio de goles:** por fecha y en toda la edición, con WITH ROLLUP, comando de SQL que hace que, además de una fila por cada grupo, la consulta devuelva filas extra con los subtotales y el total general, esto para la fila del total.
- **Canchas y turnos más utilizados:** muestra por separado los partidos jugados y los programados, además del total. Los postergados y los cancelados no cuentan.
- **Jugadores por facultad y tipo:** usa WITH ROLLUP y GROUPING(), función de SQL que permite identificar las filas de subtotales y totales generadas por WITH ROLLUP.
- **Inscriptos que nunca figuraron en una planilla:** usa NOT EXISTS.
- **Sanciones por equipo y motivo:** parte de v_sanciones y mira el color de la tarjeta que originó la sanción para separar la roja directa de la doble amarilla.
- **Porcentaje de partidos jugados, postergados y cancelados:** agrupa por estado y usa SUM() OVER() para calcular el porcentaje.

Además propusimos tres consultas propias. La de remontadas muestra los partidos que ganó un equipo que en algún momento iba perdiendo, reconstruye el marcador gol a gol con una suma acumulada ordenada por minuto. La de goles por franja muestra en qué momento del partido se hacen los goles. La de rachas muestra los últimos tres resultados de cada equipo usando ROW_NUMBER(), junto a su posición actual.

Seguridad

Usuarios por rol

El backend no se conecta a la base con el usuario root. En db/05_usuarios.sql definimos tres usuarios de MySQL, cada uno con los permisos mínimos para su función, otorgados tabla por tabla.

El usuario administrador puede leer y escribir en las tablas de la aplicación, salvo turno y auditoria_resultado, que solo puede leer. También puede ejecutar marcar_jugado y generar_fixture.

El usuario planillero puede leer lo que necesita para completar una planilla: facultades, jugadores, torneos, equipos y planteles, complejos y canchas, turnos, partidos, sanciones y algunas vistas. Solo puede escribir en planilla_partido, gol y tarjeta, y solo puede ejecutar marcar_jugado. No tiene acceso a login ni a auditoria_resultado.

El usuario auth solo puede leer la tabla login. Se usa para validar las credenciales al iniciar sesión, cuando todavía no se conoce el rol. Una vez autenticada la persona, el backend abre cada conexión con torneo_admin o con torneo_planillero, según corresponda.

Algunos casos necesitaron una solución específica. El planillero no puede hacer un update sobre partido, así que para cerrar un partido ejecuta marcar_jugado, que solo cambia su estado. De este modo puede cerrarlo, pero no alterar sus demás datos ni cancelarlo. Las sanciones automáticas las registran los triggers de la tabla tarjeta, que se ejecutan con los permisos de quien los definió, y por eso el planillero no necesita escribir en sancion_jugador. Por la misma razón, auditoria_resultado la completan solo los triggers.

Este esquema agrega un segundo control, además de la verificación de roles del backend. Si por un error en la aplicación un planillero intentara programar un partido, MySQL lo rechazaría por falta de privilegios, y el backend convertiría ese rechazo en una respuesta HTTP 403 con un mensaje.

Login y tokens

Las contraseñas de la aplicación se guardan con hash, usando bcrypt, un algoritmo que se usa para guardar contraseñas de forma segura y nunca en texto plano. Al iniciar sesión, POST /login devuelve un token firmado con PyJWT, un texto con datos adentro y una firma digital que garantiza que nadie los modificó, válido por 8 horas, que incluye el correo y el rol. Cada endpoint declara qué roles lo pueden usar.

Auditoría

En los partidos ya jugados queda registrado en auditoria_resultado cada cambio realizado, estos pudiendo ser: las altas, modificaciones y bajas de goles y tarjetas, y las altas y bajas de la planilla, con el detalle, el usuario y la fecha y hora. Los cambios de la planilla mediante update no se registran. El backend le indica a la base el correo de quien hizo el cambio, de modo que la auditoría guarda a la persona y no solo al usuario.

Otras medidas

En las consultas del backend, los datos que llegan del usuario siempre van como parámetros y nunca concatenados al texto SQL, lo que previene el ataque visto en el curso SQL injection. Los errores de MySQL se traducen a respuestas HTTP con mensajes claros, sin mostrar detalles internos. La configuración sensible se toma de variables de entorno, aunque todavía quedan valores por defecto, como se explica en las limitaciones.

Backend, frontend y pruebas

Backend

El backend está organizado en módulos chicos. En database.py está la conexión por rol, la zona horaria y el usuario de auditoría; en auth.py, el login, el token y las dependencias por rol; y en errores.py, la traducción de los errores de MySQL. Además hay un archivo por grupo de endpoints en la carpeta routers: facultades, torneos, equipos, jugadores, complejos, partidos, planillas, sanciones y reportes. FastAPI genera la documentación interactiva en /docs.

Frontend

El frontend consume la API. Tiene la pantalla de login, una barra de navegación superior que muestra las secciones según el rol, y las pantallas de posiciones, fixture, planillas, sanciones, reportes y catálogos. El planillero solo ve lo que le corresponde.

Para el diseño visual nos inspiramos en el sitio *Every Pokémon is Someone's Favorite*, porque nos gustó su estética y sentimos que se podía adaptar al sistema. Usamos *Pixelify Sans*, una tipografía pixelada, para los títulos, *DM Sans* para el texto y *DM Mono* para datos como fechas y estadísticas. A cada equipo se le asigna un color y una insignia con su sigla, que se repiten en el fixture, la tabla de posiciones y los reportes para reconocerlo más fácil. Antes de programar las pantallas armamos una referencia en Figma, que está en docs/Torneo Interfacultades — estética planilla.pdf. Las figuras a continuación muestran dos pantallas del resultado final.

Figura 2 - Pantalla de reportes

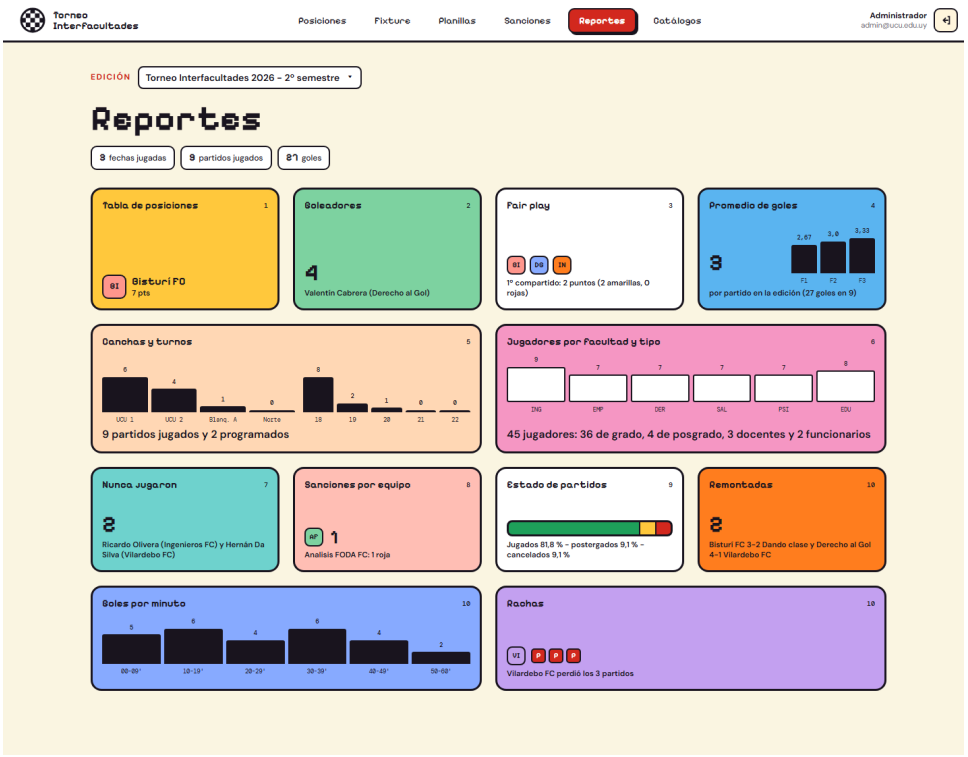
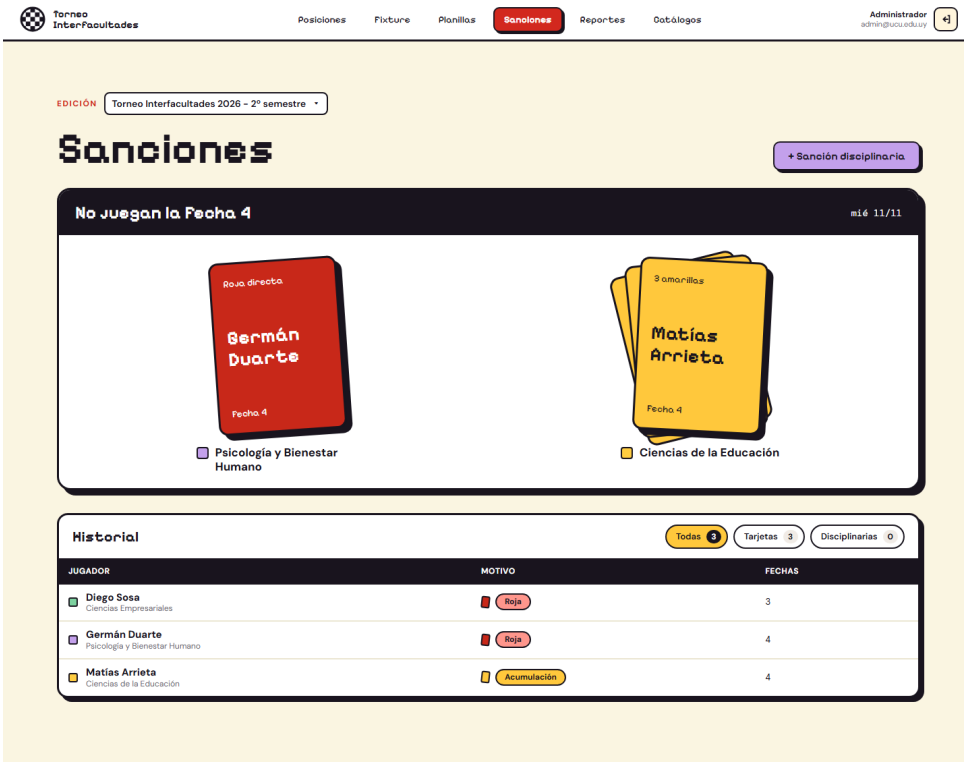


Figura 3 - Pantalla de sanciones



Pruebas automatizadas

Aunque no estaba pedido, creímos que podía ser interesante hacer algo de testing. Por lo mismo, hicimos 226 tests con pytest que corren contra un MySQL, sin simulaciones. Al comenzar la ejecución se recrea la base desde los scripts, con los datos de prueba incluidos.

En tests/db hay 146 casos que se conectan directamente a MySQL e intentan violar las reglas de la letra. También verifican los permisos de cada usuario, el fixture y los resultados de los reportes. Cada test corre dentro de una transacción que se deshace al final, para que no dependan entre sí.

En tests/api hay 80 casos que prueban el backend: login y permisos por rol, altas, bajas y modificaciones de los catálogos, partidos, planillas, sanciones, reportes y traducción de errores. Como el backend confirma cada operación, estos tests trabajan sobre ediciones propias con fechas lejanas, para no modificar los datos de prueba.

Las pruebas, además de darnos unos largos dolores de cabeza, nos ayudaron a encontrar y corregir varios errores. Entre ellos, que borrar una amarilla dejaba una sanción de más, que se podía cambiar el color de una tarjeta, marcar como jugado un partido sin planilla, iniciar un torneo con equipos incompletos o mover un partido ya jugado, y que el top 10 de goleadores ignoraba los empates. En la revisión final también encontramos que la doble amarilla dependía del orden de carga de las tarjetas, lo corregimos y agregamos tests para ese caso.

Mejoras implementadas y consideradas

Implementadas

Además de lo que pide la letra, sumamos varias cosas. Las principales son la auditoría de correcciones, la generación automática del fixture, los tres reportes propios y las sanciones que se recalculan al corregir una tarjeta.

También agregamos reglas de consistencia que no estaban pedidas, un expulsado no registra goles ni tarjetas después de su expulsión, toda edición empieza en inscripción y ningún equipo queda sin capitán una vez iniciado el torneo.

Para facilitar las pruebas, los datos de ejemplo usan fechas relativas al día de carga, así siempre hay fechas jugadas, una dentro de la ventana de 48 horas y una a futuro.

Consideradas y no implementadas

Pensamos en otras mejoras que decidimos dejar afuera. Un historial de planteles entre ediciones, con fechas de alta y baja, no lo pide la letra y complicaba las validaciones. Una tabla de fechas del torneo, con el día previsto para cada fecha, la descartamos para no duplicar información que ya se calcula desde los partidos. La revocación de tokens quedó fuera del alcance del obligatorio. Tampoco agregamos índices extra para los reportes, porque con el volumen esperado alcanzan los de las claves.

Limitaciones y vulnerabilidades conocidas

Al tener que buscar posibles riesgos, encontramos los siguientes puntos, junto con la forma en que los resolveríamos antes de usar el sistema en un entorno real.

- **Credenciales de ejemplo:** .env.example, 05_usuarios.sql, el código y Docker Compose tienen contraseñas y una SECRET_KEY de ejemplo para que el proyecto se pueda correr. Habría que eliminarlas, usar variables de entorno o un gestor de secretos y cambiar las credenciales.
- **Tráfico sin cifrar:** la API corre en HTTP local. Habría que publicarla detrás de HTTPS.

- **Tokens no revocables:** un token robado sirve hasta que vence, a las 8 horas. Además, el frontend lo guarda en localStorage, donde podría leerlo un script malicioso. Convendría usar vencimientos más cortos y tokens de refresco.
- **Fuerza bruta en el login:** no hay un límite de intentos. Habría que limitarlos por IP o por usuario.
- **Acceso como root:** quien tenga root podría alterar el esquema, incluidas las reglas.
- **Datos de prueba con fechas relativas:** si pasan más de dos días desde la carga, la fecha 3 queda fuera de la ventana de 48 horas. Hay que recrear la base antes de probar, como indica el README.
- **Partidos sin cerrar:** las 48 horas se cuentan desde que termina el partido, pero el límite solo se aplica cuando ya está marcado como jugado. Si nadie lo marca, la planilla se puede seguir modificando. Se podría marcar como jugado automáticamente.
- **Operaciones simultáneas:** varios triggers primero cuentan y después insertan, como el del máximo de jugadores. Si dos personas hicieran una carga en el mismo instante, podrían pasar un cupo. No lo probamos, y para este uso nos parece poco probable. Se resolvería bloqueando la fila del equipo durante la operación.
- **Roja y tercera amarilla en el mismo partido:** se generan las dos sanciones, pero las dos caen en la fecha siguiente, por lo que el jugador cumple una sola fecha. Se resolvería con el mismo criterio que usamos para la doble amarilla.
- **Doble amarilla como texto:** en la base, la doble amarilla se guarda con el motivo "roja" y la pantalla de sanciones la reconoce por su descripción. La descripción la genera solo el trigger y no se puede editar, pero sería más prolijo tener un motivo propio.

Conclusiones

Lo que más nos sirvió fue poner las reglas en la base de datos. Muchas se pudieron resolver solo con el esquema, usando claves compuestas y columnas generadas, y eso nos ahorró triggers y nos mostró cuánto se puede lograr con un buen diseño antes de escribir lógica.

Los tests fueron otra parte importante. Aunque no estaban pedidos, encontraron errores que a mano no habríamos visto, y nos dieron tranquilidad para cambiar reglas sobre el final. También aprendimos que una regla que parece simple puede complicarse: la doble amarilla nos obligó a pensar qué pasa si los datos se cargan en otro orden, si se corrigen después o si coincide con otra sanción.

Si volviéramos a empezar, arrancaríamos antes con el frontend, porque quedó concentrado en las últimas semanas y varias reglas terminaron de ajustarse cuando ya estaban las pantallas. De todas formas, el sistema cumple lo que pide la letra y las limitaciones que quedan están identificadas.

Bitácora

Este es un registro de cómo nos fuimos organizando. Además, utilizamos un tablero en Trello que está adjunto en el README del repositorio si se desea visualizar, y el detalle de cada cambio está en el historial de commits del repositorio.

Semana del 9 al 15 de octubre. El 9/10 recibimos la letra, la leímos en equipo y repartimos las tareas. Analizamos los requerimientos e hicimos un primer borrador de las tablas y de las reglas del torneo.

Semana del 16 al 22 de octubre. El 16/10 creamos la primera versión de la base de datos y las primeras reglas automáticas, y armamos el diagrama y el README. El 17 y el 18/10 definimos los tipos de usuario y decidimos dónde controlar cada regla de la letra.

Semana del 23 al 29 de octubre. Revisamos el diseño de la base y agregamos los estados del torneo, el plazo de 48 horas y las sanciones automáticas. Sumamos la generación del fixture, el cierre de partidos, los reportes y los usuarios con permisos por rol, y cargamos datos de prueba. Armamos la primera versión del backend y del frontend. Escribimos pruebas automáticas (144 en ese momento), preparamos Docker y corregimos los errores que encontraron las pruebas.

Semana del 30 de octubre al 3 de noviembre. El 30/10 hicimos la entrega de avance y el 31/10 corregimos el README. El 2/11 diseñamos el menú de navegación y la tabla de posiciones. El 3/11 definimos las referencias de diseño del frontend, mejoramos la pantalla de posiciones y diseñamos las pantallas de inicio de sesión y del fixture.

Semana del 4 al 6 de noviembre. El 4/11 agregamos la pantalla de jugadores, los permisos por rol en el frontend y el logo, completamos los catálogos, terminamos la carga de planillas, goles y tarjetas, y agregamos la sección de reportes. El 5/11 rediseñamos el frontend y sumamos reglas nuevas: ningún equipo queda sin capitán, no se inscriben equipos con el torneo empezado y un expulsado no puede tener goles ni tarjetas después de su expulsión. Ese mismo día agregamos la doble amarilla, corregimos los casos de carga en otro orden, decidimos sumar las sanciones cuando coinciden y llegamos a 200 tests. También ajustamos la estética de los reportes y de los mensajes de la aplicación, y revisamos el informe y el README. El 6/11 es la entrega final y el 13/11, la defensa.

Reparto del trabajo

Todos participamos en las decisiones de diseño. Maximiliano trabajó sobre todo en la base de datos, las pruebas, el backend, los reportes y el rediseño del frontend. Trinidad trabajó en el backend y sus pruebas, en los catálogos que faltaban y en la regla de doble amarilla. Paulina diseñó la barra de navegación, la tabla de posiciones, el login y el fixture, e implementó la gestión de planillas, goles y tarjetas, además de colaborar con el backend.

Reflexiones finales

Este obligatorio fue uno de los trabajos más desafiantes de la carrera. La letra tenía muchas reglas que se cruzaban entre sí, y lograr que todas se cumplieran al mismo tiempo, sin importar el orden en que se cargaran los datos, nos llevó a repensar el diseño más de una vez.

También aprendimos la importancia de las pruebas. Muchos errores que parecían imposibles, como sanciones duplicadas o partidos que se podían modificar cuando no correspondía, aparecieron recién cuando escribimos tests que intentaban romper las reglas a propósito. Gracias a eso pudimos hacer cambios con confianza, sabiendo que no rompíamos lo que ya funcionaba.

Estamos muy conformes con el resultado. Sin duda fue un proyecto que nos permitió combinar lo aprendido en varias materias de la carrera, como Bases de Datos, Desarrollo Web y Mobile, Programación II, entre otras y fue muy gratificante ver todo eso funcionando en conjunto en una aplicación. Agradecemos a los profesores por el acompañamiento durante el curso.

Referencias

bcrypt. (s. f.). *bcrypt* [Software]. PyPI. https://pypi.org/project/bcrypt/

Docker. (s. f.). *Docker Compose documentation*. https://docs.docker.com/compose/

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Every Pokémon is Someone's Favorite. (s. f.). https://favoritepokemon.vercel.app/

FastAPI. (s. f.). *FastAPI documentation*. https://fastapi.tiangolo.com/

Oracle. (s. f.-a). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

Oracle. (s. f.-b). *MySQL Connector/Python developer guide*. https://dev.mysql.com/doc/connector-python/en/

Pydantic. (s. f.). *Pydantic documentation*. https://docs.pydantic.dev/

PyJWT. (s. f.). *PyJWT documentation*. https://pyjwt.readthedocs.io/

pytest. (s. f.). *pytest documentation*. https://docs.pytest.org/

React. (s. f.). *React documentation*. https://react.dev/

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2019). *Database system concepts* (7.ª ed.). McGraw-Hill.

Universidad Católica del Uruguay. (2026). *Material del curso Bases de Datos I*.

Vite. (s. f.). *Vite documentation*. https://vite.dev/

Declaración del uso de inteligencia artificial

Para la elaboración de este trabajo utilizamos herramientas de inteligencia artificial, principalmente Claude (Anthropic) y ChatGPT (OpenAI), como apoyo durante el desarrollo del proyecto. Las usamos para resolver dudas, consultar posibles soluciones, colaborar en el desarrollo del backend y frontend, configurar el entorno con Docker y revisar algunos aspectos de la documentación.

Las sugerencias obtenidas fueron revisadas, probadas y ajustadas por nosotros antes de incorporarlas al proyecto. Como equipo, asumimos la responsabilidad final sobre el código, las decisiones tomadas y el contenido del informe.